

Engenharia de Software II

André L. D. Rossi

Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências (FC) / Departamento de Computação (DCo)
Bauru, SP - Brasil

1. Estratégias e teste de software
2. Uma abordagem estratégica do teste de software
3. Estratégias de teste para software
4. Test Driven Development
5. Teste de validação
6. Teste de sistema
7. A arte da depuração

Estratégias e teste de software

O que é:

O software é testado para **revelar erros** cometidos inadvertidamente quando foi projetado e construído.

Muitas questões sobre teste de software são respondidas quando desenvolvemos uma **estratégia de teste de software**.

A **estratégia de teste** de software define os **passos** para realização dos testes, **quando** esses passos serão executados e **quanto** trabalho, **tempo** e **recursos** serão necessários.

Portanto, qualquer estratégia de teste deve incorporar:

- planejamento dos testes;
- projeto de casos de teste;
- execução dos testes;
- coleta e avaliação dos dados resultantes.

Uma estratégia de teste de software deve ser flexível o bastante para promover uma estratégia de teste personalizada.

Uma abordagem estratégica do teste de software

Uma abordagem estratégica do teste de software

Teste é um conjunto de atividades que podem ser planejadas com antecedência e executadas sistematicamente.

Por essa razão, deverá ser definido, para o processo de software, um **modelo para o teste** – um conjunto de etapas no qual podem ser empregadas técnicas específicas de projeto de caso de teste e métodos de teste.

Uma abordagem estratégica do teste de software

Uma estratégia de teste de software deve acomodar testes de baixo e alto nível.

Testes de baixo nível: necessários para verificar se um pequeno segmento de código fonte foi implementado corretamente.

Testes de alto nível: validam as funções principais do sistema de acordo com os requisitos do cliente.

Uma estratégia deve fornecer diretrizes para o profissional e uma série de metas para o gerente.

Verificação e validação

O teste de software é um elemento de um tema mais amplo, muitas vezes conhecido como verificação e validação (V&V).

Verificação: refere-se ao conjunto de tarefas que garantem que o software implementa corretamente uma função específica.

Validação: refere-se a um conjunto de tarefas que asseguram que o software foi criado e pode ser rastreado segundo os requisitos do cliente.

Boehm¹ define de outra maneira:

- **Verificação:** “Estamos criando o produto corretamente?”
- **Validação:** “Estamos criando o produto certo?”

¹Boehm, B., Software Engineering Economics, Prentice Hall, 1981.

Verificação e validação

A verificação e a validação incluem uma ampla gama de atividades de SQA (*software quality assurance* – garantia da qualidade de software):

- revisões técnicas,
- auditorias de qualidade e configuração,
- monitoramento de desempenho,
- simulação,
- estudo de viabilidade,
- revisão de documentação,
- revisão de base de dados,
- análise de algoritmo,

- teste de desenvolvimento,
- teste de usabilidade,
- teste de qualificação,
- teste de aceitação e
- teste de instalação.

Embora a aplicação de teste tenha um papel extremamente importante em V&V, muitas outras atividades também são necessárias.

O teste proporciona o último elemento a partir do qual a **qualidade pode ser estimada** e, mais pragmaticamente, os erros podem ser descobertos.

“Você não pode testar qualidade. Se a qualidade não está lá antes de um teste, ela não estará lá quando o teste terminar”.

A qualidade é incorporada ao software por meio do processo de engenharia de software.

Organizando o teste de software

Organizando o teste de software

Para todo projeto de software, há um conflito de interesses inerente que ocorre logo que o teste começa:

As pessoas que criaram o software são agora convocadas para testá-lo.

Isso parece essencialmente inofensivo; afinal, quem conhece melhor o programa do que os seus próprios desenvolvedores?

Infelizmente, esses mesmos desenvolvedores têm **interesse em demonstrar que o programa é isento de erros** e funciona de acordo com os requisitos do cliente.

Organizando o teste de software

Assim, os testes são projetados e executados para demonstrar que o programa funciona, em vez de descobrir os **erros**.

Infelizmente, no entanto, os **erros estão presentes**.

Se o engenheiro de software não os encontrar, o cliente encontrará!

Frequentemente, há muitas **noções incorretas** que podem ser inferidas a partir da discussão apresentada:

- que o desenvolvedor de software não deve fazer nenhum teste;
- que o software deve ser “atirado aos leões”, ou seja, entregue a estranhos que realizarão testes implacáveis,
- que os testadores se envolvem no projeto somente no início das etapas do teste.

Organizando o teste de software

O **desenvolvedor** do software é sempre **responsável** pelo **teste das unidades** individuais (componentes) do programa, garantindo que cada uma execute a função para a qual foi projetada.

Em **muitos casos**, o **desenvolvedor** também faz parte do **teste de integração** – uma etapa de teste que leva à construção (e teste) da arquitetura completa do software.

Somente depois que a arquitetura do software está concluída é que o grupo de teste independente se envolve.

Organizando o teste de software

O papel de um **grupo independente de teste** (ITG, *independent test group*) é remover problemas inerentes associados ao fato de deixar o criador testar aquilo que ele mesmo criou.

O teste independente remove o conflito de interesses que, de outra forma, poderia estar presente. Afinal, o pessoal de ITG é pago para encontrar erros.

Enquanto o teste está sendo realizado, o desenvolvedor deve estar disponível para corrigir os erros encontrados.

Organizando o teste de software

O ITG faz parte da equipe de desenvolvimento de software, pois planeja e especifica procedimentos de teste) durante o projeto inteiro.

No entanto, em muitos casos o ITG se reporta à organização de garantia da qualidade do software, adquirindo, assim, um grau de independência que poderia não ser possível se fizesse parte da equipe de engenharia de software.

Estratégia de teste de software – visão global

Estratégia de teste de software – visão global

O processo de desenvolvimento e teste do software podem ser visto como uma espiral.

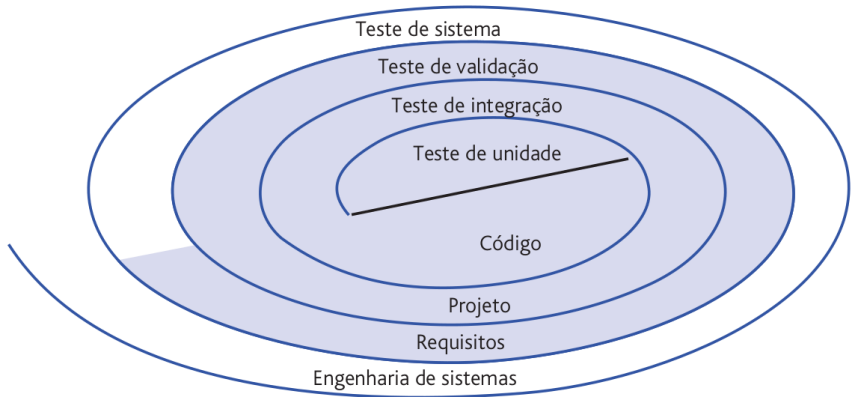


Figura 1: Estratégia de teste.

Inicialmente, os testes **focalizam cada componente individualmente**, garantindo que ele funcione adequadamente como uma unidade.

Por isso o nome **teste de unidade**.

O teste de unidade usa intensamente técnicas de teste, com **caminhos específicos na estrutura de controle** de um componente para garantir a cobertura completa e a máxima detecção de erros.

Em seguida, o **componente deve ser integrado** para formar o pacote de software completo.

O **teste de integração** cuida de problemas associados a aspectos duais de verificação e construção de programa.

Técnicas de projeto de casos de teste que **focalizam entradas e saídas** são mais predominantes durante a integração, embora técnicas que usam caminhos específicos de programa possam ser utilizadas para segurança dos principais caminhos de controle.

Depois que o software foi integrado (construído), é executada uma série de testes de ordem superior.

Os critérios de validação (estabelecidos durante a análise de requisitos) devem ser avaliados.

O **teste de validação** proporciona a garantia final de que o software satisfaz a todos os requisitos funcionais, comportamentais e de desempenho.

A última etapa de teste de ordem superior **extrapola os limites da engenharia de software**, entrando em um contexto mais amplo de engenharia de sistemas de computadores.

O software, uma vez validado, deve ser combinado com outros elementos do sistema (por exemplo, hardware, pessoas, base de dados).

O **teste de sistema** verifica se todos os elementos se combinam corretamente e se a função/desempenho global do sistema é obtida.

Estratégia de teste de software – visão global

As etapas estão ilustradas na figura:

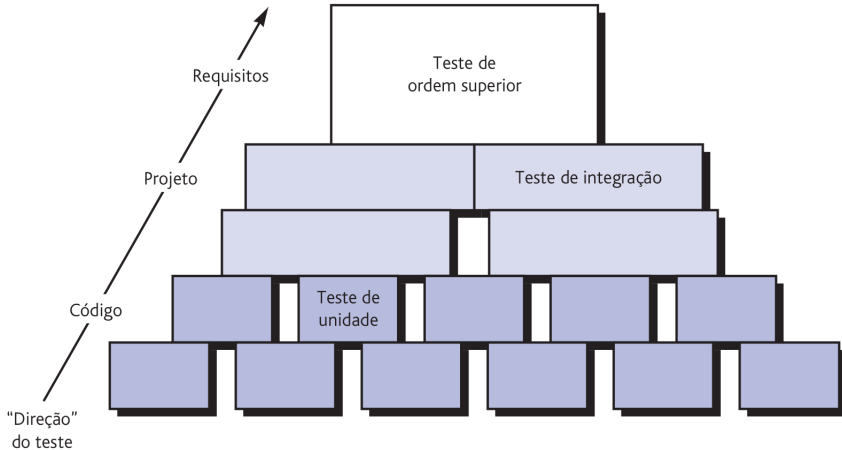


Figura 2: Estratégia de teste.

Critérios para conclusão do teste

Critérios para conclusão do teste

Uma questão clássica surge todas as vezes que se discute teste de software:

“Quando podemos dizer que terminamos os testes – como podemos saber que já testamos o suficiente?”.

Infelizmente, não há uma resposta definitiva para essa pergunta, mas há algumas respostas pragmáticas e algumas tentativas iniciais e empíricas.

Critérios para conclusão do teste

Uma resposta é:

- “O teste nunca termina; o encargo simplesmente passa do engenheiro de software para o usuário” .

Critérios para conclusão do teste

Todas as vezes que o usuário executa o programa no computador, o programa está sendo testado.

Esse fato destaca a importância de outras atividades de garantia da qualidade do software.

Critérios para conclusão do teste

Outra resposta (um tanto cínica, mas, ainda assim, exata) é:

“O teste acaba quando o tempo ou o dinheiro acabam”.

Critérios para conclusão do teste

O fato é que é necessário um **critério mais rigoroso** para determinar quando já foram executados testes em número suficiente.

Coletando **métricas** durante o teste do software e utilizando **modelos estatísticos** existentes, é possível desenvolver diretrizes significativas para responder à questão:

“Quando terminamos o teste?”.

Cobertura de Testes

Cobertura de testes é uma métrica que ajuda a definir o número de testes que precisamos escrever para um programa.

Ela mede o percentual de comandos de um programa que são cobertos por testes:

$$\text{cobertura de testes} = \frac{\text{número de comandos executados pelos testes}}{\text{total de comandos do programa}}$$

Cobertura de testes

Portanto, se os testes contemplam todas os comandos da unidade, temos 100% de cobertura, conforme mostrado na Figura a seguir:

```
public class Stack<T> {  
    private ArrayList<T> elements = new ArrayList<T>();  
    private int size = 0;  
  
    public int size() {  
        return size;  
    }  
  
    public boolean isEmpty(){  
        return (size == 0);  
    }  
  
    public void push(T elem) {  
        elements.add(elem);  
        size++;  
    }  
  
    public T pop() throws EmptyStackException {  
        if (isEmpty())  
            throw new EmptyStackException();  
        T elem = elements.get(size-1);  
        size--;  
        return elem;  
    }  
}
```

Figura 3: Teste com 100% de cobertura.

Suponha agora que não tivéssemos implementado `testEmptyStackException`.

Isto é, não iríamos testar o levantamento de uma exceção pelo método `pop()`, quando chamado com uma pilha vazia.

Nesse caso, a cobertura dos testes cairia para 92.9%, como mostrado na Figura 31.

Cobertura de testes

```
public class Stack<T> {  
    private ArrayList<T> elements = new ArrayList<T>();  
    private int size = 0;  
  
    public int size() {  
        return size;  
    }  
  
    public boolean isEmpty(){  
        return (size == 0);  
    }  
  
    public void push(T elem) {  
        elements.add(elem);  
        size++;  
    }  
  
    public T pop() throws EmptyStackException {  
        if (isEmpty())  
            throw new EmptyStackException();  
        T elem = elements.get(size-1);  
        size--;  
        return elem;  
    }  
}
```

Figura 4: Em Java, ferramentas de cobertura de testes trabalham instrumentando os bytecodes gerados pelo compilador da linguagem. O código, após compilado, possui 52 instruções cobertas por testes de unidade, de um total de 56 instruções. Portanto, sua cobertura é $52 / 56 = 92.9\%$.

Qual a cobertura de testes ideal?

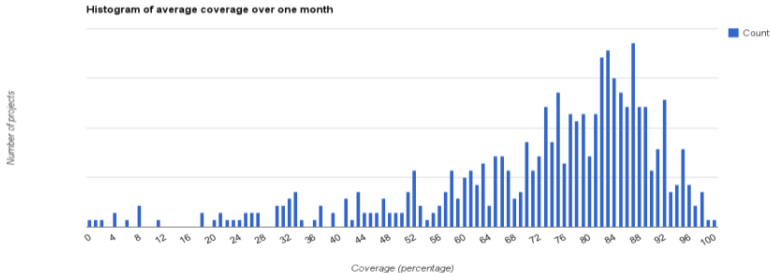
Depende do projeto, mas não precisa ser 100%.

Segundo alguns autores, no mínimo 60%.

Entre 80% e 90% seria uma recomendação, de acordo com alguns desenvolvedores da Google².

²Artigo e Apresentação

Cobertura de testes



The median is 78%, the 75th percentile 85% and 90th percentile 90%.

Estratégias de teste para software

Muitas estratégias podem ser utilizadas para testar um software.

Em um dos extremos, pode-se aguardar até que o sistema esteja totalmente construído e então realizar os testes no sistema completo na esperança de encontrar erros.

Essa abordagem, embora atraente, simplesmente não funciona.

Ela resultará em um software cheio de erros que desapontará a todos os envolvidos.

No outro extremo, você pode executar testes diariamente, sempre que uma parte do sistema for construída.

Uma estratégia de teste escolhida por muitas equipes de software está entre os dois extremos.

Ela assume uma visão incremental do teste, começando com o teste das unidades individuais de programa, passando para os testes destinados a facilitar a integração de unidades (às vezes diariamente) e culminando com testes que usam o sistema concluído.

Teste de unidade

O teste de unidade focaliza o esforço de verificação na menor unidade de projeto do software – o componente ou módulo de software.

Usando como guia a descrição de projeto no nível de componente, caminhos de controle importantes são testados para descobrir erros dentro dos limites do módulo.

A complexidade relativa dos testes e os erros que revelam são limitados pelo escopo restrito estabelecido para o teste de unidade.

Esse teste enfoca a **lógica interna de processamento e as estruturas de dados** dentro dos limites **de um componente**.

Esse tipo de teste pode ser conduzido em **paralelo** para diversos componentes.

Teste de unidade

Os testes de unidade estão ilustrados esquematicamente na Figura 5:

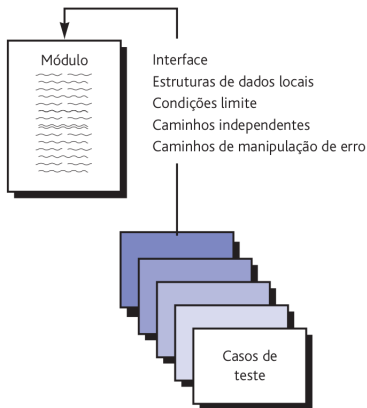


Figura 5: Teste de unidade.

O fluxo de dados por meio da **interface de um componente** é testado antes de iniciar qualquer outro teste.

Se os dados não entram e saem corretamente, todos os outros testes são discutíveis.

Além disso, estruturas de dados locais deverão ser ensaiadas, e o impacto local sobre dados globais deve ser apurado (se possível) durante o teste de unidade.

O **teste seletivo de caminhos de execução** é uma tarefa essencial durante o teste de unidade.

Casos de teste devem ser projetados para descobrir erros devido a computações errôneas, comparações incorretas ou fluxo de controle inadequado.

O **teste de fronteira** é uma das tarefas mais importantes do teste de unidade.

As falhas nas fronteiras do software são mais suscetíveis de ocorrerem.

Isto é, os erros frequentemente ocorrem quando o n -ésimo elemento de um conjunto n -dimensional é processado, quando a i -ésima repetição de um laço com i passadas é chamada ou quando o valor máximo ou mínimo permitido é encontrado.

Casos de teste que utilizam estrutura de dados, fluxo de controle e valores de dados logo abaixo, iguais ou logo acima dos máximos e mínimos têm grande possibilidade de descobrir erros.

O teste de unidade normalmente é considerado um auxiliar para a etapa de codificação.

O projeto dos testes de unidade pode ocorrer antes da codificação começar ou depois que o código-fonte tiver sido gerado.

Cada caso de teste deverá ser acoplado a um **conjunto de resultados esperados**.

Quando consideramos o software orientado a objetos, o conceito de unidades se modifica.

O encapsulamento controla a definição de classes e objetos.

Isso significa que cada classe e cada instância de uma classe (objeto) empacotam atributos (dados) e as operações que manipulam esses dados.

Uma **classe** encapsulada é usualmente o **foco do teste de unidade**.

No entanto, **operações (métodos)** dentro da classe **são as menores unidades testáveis**.

O teste de classe para software orientado a objetos é equivalente ao teste de unidade para software convencional.

Teste de unidade para software convencional: tende a focalizar o detalhe algorítmico de um módulo e os dados que fluem por meio da interface do módulo

Teste de classe para software orientado a objetos: é controlado pelas operações encapsuladas pela classe e o comportamento de estado da classe.

ATIVIDADE PRÁTICA

Teste de integração

Um novato no mundo do software pode levantar uma questão aparentemente legítima quando todos os módulos tiverem passado pelo teste de unidade:

“Se todos funcionam individualmente, por que você duvida que funcionem quando estiverem juntos?”.

O **teste de integração** é uma técnica sistemática para construir a arquitetura de software a partir dos componentes testados em unidade, ao mesmo tempo em que se realizam testes para **descobrir erros associados às interfaces**.

Possíveis erros que podem ocorrer :

- Dados podem ser perdidos através de uma interface;
- Um componente pode ter um efeito inesperado ou adverso sobre outro;
- Subfunções, quando combinadas, podem não produzir a função principal desejada;
- Imprecisão aceitável individualmente pode ser amplificada em níveis não aceitáveis;
- Estruturas de dados globais podem apresentar problemas.

Teste de integração

Muitas vezes há uma tendência de tentar a integração não incremental; isto é, construir o programa usando uma abordagem “big bang”.

Todos os componentes são combinados, e o programa inteiro é testado como um todo.

E, normalmente, o resultado é o **caos**!

Os erros são encontrados, mas a correção é difícil porque o **isolamento das causas** é complicado pela vasta extensão do programa inteiro.

Integração descendente

Teste de integração descendente (top-down) é uma abordagem incremental para a construção da arquitetura de software.

Os módulos são integrados deslocando-se para baixo por meio da hierarquia de controle, começando com o módulo de controle principal (programa principal).

Módulos subordinados ao módulo de controle principal são incorporados à estrutura de uma maneira **primeiro-em-profundidade ou primeiro-em-largura** (depth-first ou breadth-first).

Teste de integração

Na figura, a integração **primeiro-em-profundidade** integra todos os componentes em um caminho de controle principal da estrutura do programa.

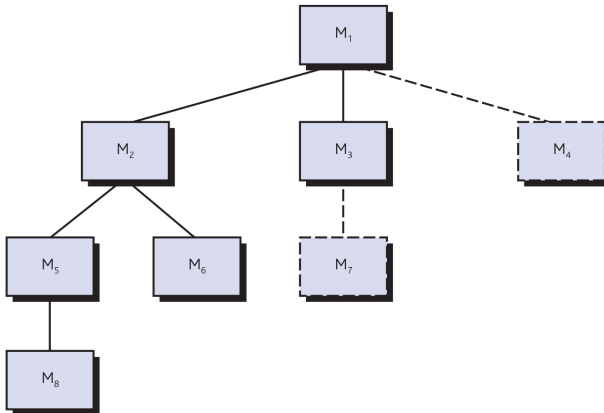


Figura 6: Integração descendente.

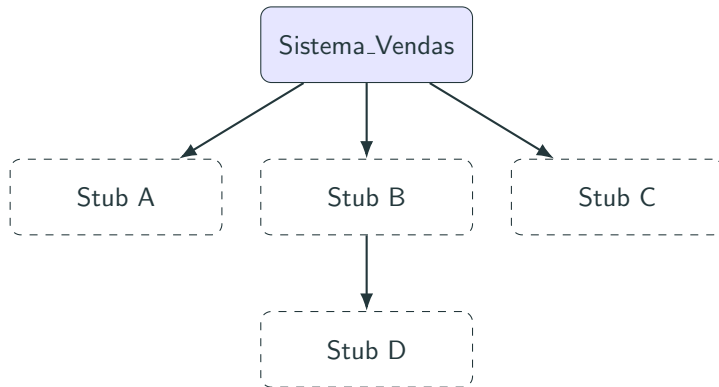
A integração **primeiro-em-largura** incorpora todos os componentes diretamente subordinados a cada nível, movendo-se através da estrutura horizontalmente.

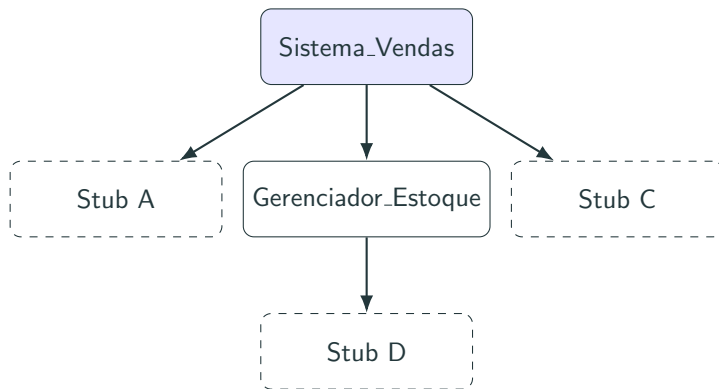
Pela figura, os componentes M2, M3 e M4 seriam integrados primeiro. Em seguida vem o próximo nível de controle, M5, M6 e assim por diante.

Entretanto, a seleção de um caminho principal é, de certa forma, arbitrária e depende das características específicas da aplicação!

O processo de integração é executado em uma série de cinco passos:

1. O módulo de controle principal é o ponto de partida e todos os componentes diretamente subordinados a ele são inicialmente substituídos por pseudocontrolados (*stubs*).
2. Dependendo da abordagem de integração (profundidade ou largura), os *stubs* são substituídos, um de cada vez, pelos componentes reais correspondentes.
3. Os testes são realizados à medida que cada componente é integrado ao sistema.
4. O teste de regressão (discutido mais adiante) pode ser executado para garantir que não tenham sido introduzidos novos erros.





Se for selecionada a **integração em profundidade**, uma função completa do software pode ser implementada e demonstrada.

A demonstração antecipada da capacidade funcional é um gerador de confiança para todos os envolvidos no programa.

Integração ascendente

O teste de integração ascendente (*bottom-up*), como o nome diz, começa a construção e o teste com módulos atômicos (isto é, componentes nos níveis mais baixos na estrutura do programa).

Como os componentes são integrados de baixo para cima, a funcionalidade proporcionada por componentes subordinados a determinado nível está sempre disponível, e a necessidade de *stubs* (pseudocontrolados) é eliminada.

Teste de integração

Uma estratégia de integração ascendente pode ser implementada com os seguintes passos:

1. Componentes de baixo nível são combinados em agregados (*clusters*, também chamados de *builds*, construções) que executam uma subfunção específica de software.
2. Um *driver* (pseudocontrolador) de teste é escrito para coordenar entrada e saída do caso de teste.
3. O agregado é testado.
4. Os *drivers* são removidos, e os agregados são combinados movendo-se para cima na estrutura do programa.

Teste de integração

A integração segue o padrão ilustrado na figura:

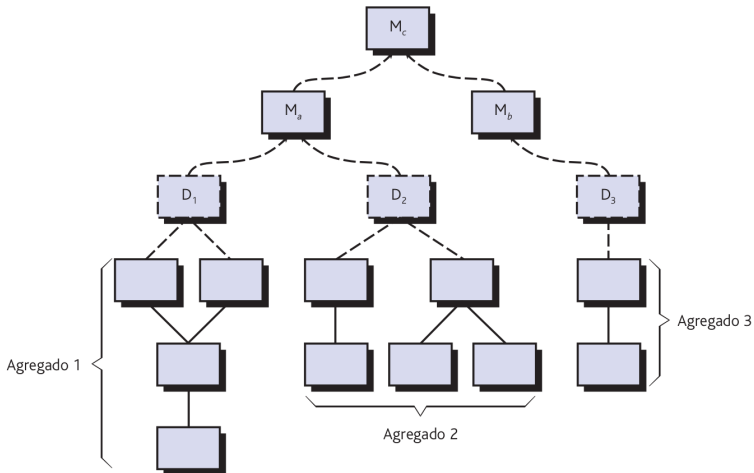


Figura 7: Integração ascendente.

Os componentes são combinados para formar os agregados (clusters) 1, 2 e 3.

Cada um dos agregados é testado usando um *driver* (mostrado como um bloco tracejado).

Componentes nos agregados 1 e 2 são subordinados a Ma.

Os pseudocontroladores D1 e D2 são removidos, e os agregados fazem interface diretamente com Ma.

De forma semelhante, o pseudocontrolador D3 para o agregado 3 é removido antes da integração com o módulo Mb.

Ma e Mb serão ambos finalmente integrados ao componente Mc e assim por diante.

À medida que a integração se move para cima, a necessidade de pseudocontroladores de testes separados diminui.

Na verdade, se os níveis descendentes da estrutura do programa forem integrados de cima para baixo, o número de pseudocontroladores pode ser bastante reduzido, e a integração de agregados fica bastante simplificada.

Teste de Regressão

Cada vez que um novo módulo é acrescentado como parte do teste de integração, o software muda.

Novos caminhos de fluxo de dados são estabelecidos, podem ocorrer novas entradas e saídas, e nova lógica de controle é chamada.

Os efeitos colaterais associados a essas alterações podem causar problemas em funções que antes funcionavam corretamente.

No contexto de uma estratégia de teste de integração, o **teste de regressão** é a **reexecução** do mesmo **subconjunto de testes** que já foram executados, para assegurar que as alterações não tenham propagado efeitos colaterais indesejados.

O teste de regressão ajuda a garantir que as alterações (devido ao teste ou por outras razões) não introduzam comportamento indesejado ou erros adicionais.

O teste de regressão pode ser executado manualmente, reexecutando um subconjunto de todos os casos de teste ou usando ferramentas automáticas de captura/reexecução.

Ferramentas de captura/reexecução permitem que o engenheiro de software capture casos de teste e resultados para reexecução e comparação subsequente.

O conjunto de teste de regressão (o subconjunto de testes a serem executados) contém três classes diferentes de casos de teste:

- Uma amostra representativa dos testes que usam todas as funções do software.
- Testes adicionais que focalizam as funções de software que podem ser afetadas pela alteração.
- Testes que focalizam os componentes do software que foram alterados.

À medida que o teste de integração progride, o número de testes de regressão pode crescer muito.

Portanto, o conjunto de testes de regressão deve ser projetado de forma a incluir somente aqueles testes que tratam de uma ou mais classes de erros em cada uma das funções principais do programa.

Teste de Fumaça

Teste fumaça

Teste fumaça é uma abordagem de teste de integração usada frequentemente quando produtos de software são desenvolvidos.

É projetado como um mecanismo de marca-passo para projetos com prazo crítico, permitindo que a equipe de software avalie o projeto frequentemente.

A frequência diária dos testes dá, tanto aos gerentes quanto aos profissionais, uma ideia realística do progresso do teste de integração.

McConnell³ descreve o teste fumaça da seguinte maneira:

O teste fumaça deve usar o sistema inteiro de fim-a-fim. Ele não precisa ser exaustivo, mas deve ser capaz de expor os principais problemas.

fumaça deve ser bastante rigoroso, de forma que, se a construção passar, você pode supor que ele é estável o suficiente para ser testado mais rigorosamente.

³McConnell, S., "Best Practices: Daily Build and Smoke Test", IEEE Software, vol. 13, no. 4, July 1996, pp. 143–144.

O teste fumaça proporciona muitos benefícios quando aplicado a projetos de engenharia de software complexos e de prazo crítico:

- *O risco da integração é minimizado.* Como os testes fumaça são feitos diariamente, as incompatibilidades e outros erros bloqueadores são descobertos logo.
- *O diagnóstico e a correção dos erros são simplificados.* Como todas as abordagens de teste de integração, os erros descobertos durante o teste fumaça provavelmente estarão associados aos “novos incrementos do software”.
- *É mais fácil avaliar o progresso.* A cada dia que passa, uma parte maior do software já está integrada e é demonstrado que funciona.

Artefatos do teste de integração.

Um plano global para integração do software e uma descrição dos testes específicos são documentados em uma Especificação de Teste. Esse artefato incorpora um plano de teste e um documento de teste e torna-se parte da configuração do software.

O teste é dividido em fases e construções que tratam de características funcionais e comportamentais específicas do software.

Cada uma dessas fases de teste de integração representa uma categoria funcional ampla dentro do software.

O ambiente e os recursos de teste são descritos. Por exemplo:

- Configurações de hardware não usuais
- Simuladores
- Ferramentas ou técnicas especiais de teste

Em seguida, é descrito o procedimento de teste detalhado necessário para realizar o plano de teste.

São descritas a ordem de integração e os testes correspondentes em cada etapa de integração.

É incluída também uma lista de todos os casos de teste (anotados para referência subsequente) e dos resultados esperados.

Relatório de Teste: histórico dos resultados reais do teste, problemas ou peculiaridades é registrado em um que pode ser anexado à Especificação de Teste, se desejado.

Essas informações podem ser vitais durante a manutenção do software.

Test Driven Development

Por que devemos escrever testes de unidade? Algumas das vantagens de se escrever testes de unidade são:

- Diminuir a quantidade de bugs do sistema;
- Diminuir a necessidade de testes manuais;
- Desenvolver uma arquitetura mais eficiente;
- Facilitar o refactoring e aumentar a confiança.

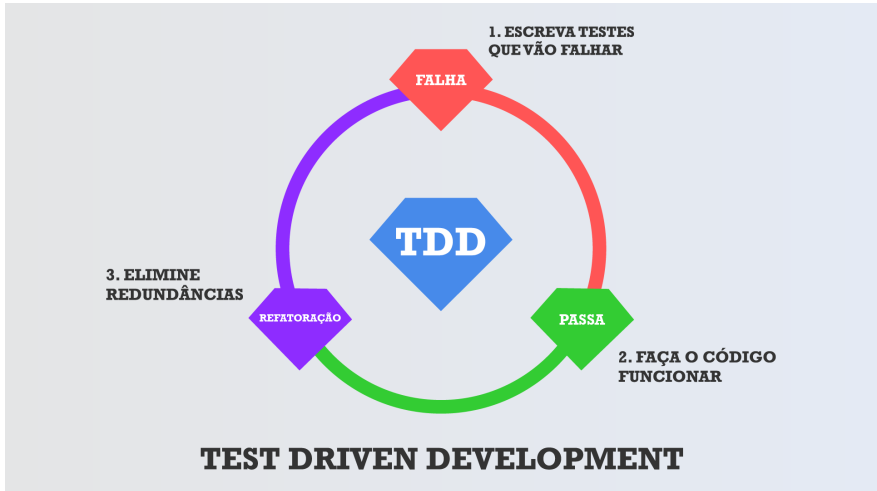
O TDD (Test Driven Development) foi desenvolvido na metodologia XP (Extreme Programming) por Kent Beck, um engenheiro de software signatário do Manifesto Ágil.

Test Driven Development

Basicamente significa escrever nossos testes de unidade antes do código de produção.

O TDD é a prática repetida de pequenos ciclos divididos em três fases: Red (failing test), Green (passing test), Blue (Refactoring).

Test Driven Development



Assim repetimos este ciclo inúmeras vezes para criar métodos, adicionar argumentos, retornos, comportamentos extras, etc. . . Nada deve ser escrito sem antes termos um teste falhando.

Um dos benefícios do TDD é fazer com que lidemos com pequenos pedaços de código de cada vez através de ciclos repetitivos de desenvolvimento.

Para garantir que o fluxo está correto existem 3 leis para nos auxiliar. São as 3 leis do TDD. Essas leis nos forçam a escrever código escalável e desacoplado.

Lei 1: Você não pode escrever nenhum código fonte, antes de escrever uma especificação de teste que falhe.

Isso quer dizer basicamente, o principal e o óbvio para a metodologia, nenhum código pode ser criado, ou classes, ou interfaces, ou seja nada, antes que a especificação do testes seja escrita e falhe, que obviamente irá falhar porque não tem nenhum código para ser validado.

Lei 2: Você não pode escrever mais do que um teste para falhar (e não compilar é falhar).

Muitas vezes já queremos adiantar todas as possíveis soluções e cenários plausíveis para garantir que o que iremos desenvolver irá cobrir os testes e atender os resultados esperados. Porém, primeiro deve-se escrever uma especificação de teste, essa especificação falhará, você então irá codificar para passar no teste e só depois você poderá escrever uma nova especificação de teste.

Dessa forma a cada nova especificação, codificação, estará sendo criado uma pilha de testes que continuarão sendo validados sobre o que está sendo desenvolvimento com qualidade e sem quebras.

Lei 3: Você não pode escrever código fonte mais do que o suficiente para passar no teste que está falhando.

Essa talvez seja a regra mais difícil de executar, porque como programador é natural durante o processo de implementação, já tentamos adiantar passos e prever condicionais, situações e comportamentos, porque dessa forma conseguimos escrever códigos que aceleram a implementação final.

Porém, a metodologia nos obriga a dar um passo de cada vez para que possamos ter ciência e principalmente cobertura de todo o processo, com isso fica bem claro que o código fonte que devemos implementar tem que ter o propósito de fazer o teste ser passado e somente isso.

Teste de validação

O teste de validação começa quando o teste de integração termina e o software está completamente montado como um pacote.

No nível de validação ou de sistema, a distinção entre diferentes categorias de software desaparece. O teste focaliza ações visíveis ao usuário e saídas do sistema reconhecíveis pelo usuário.

A validação pode ser definida de várias maneiras, mas uma definição simples (embora rigorosa) é que a validação tem sucesso quando o software funciona de uma maneira que pode ser razoavelmente esperada pelo cliente.

Nesse ponto, um desenvolvedor de software veterano pode protestar:

- “Quem ou o que é o árbitro para decidir o que são expectativas razoáveis?”.

Se uma **Especificação de Requisitos de Software** foi desenvolvida, ela descreve todos os atributos do software visíveis ao usuário e contém uma seção denominada **Critérios de Validação** que forma a base para uma abordagem de teste de validação.

Critérios de teste de validação

A validação de software é obtida por meio de uma série de testes que demonstram conformidade com os requisitos.

Critérios de teste de validação

Um **plano de teste** descreve as classes de testes a serem realizados.

Um **procedimento de teste** define casos de teste específicos destinados a garantir que:

- todos os requisitos funcionais sejam satisfeitos
- todas as características comportamentais sejam obtidas
- todo o conteúdo seja preciso e adequadamente apresentado
- todos os requisitos de desempenho sejam atendidos
- a documentação esteja correta
- outros requisitos sejam cumpridos, como transportabilidade, compatibilidade, recuperação de erro, facilidade de manutenção

Se for descoberto um desvio em relação à especificação, é criada uma lista de deficiências.

Deve ser estabelecido um método para solucionar deficiências (aceitável para os envolvidos).

Testes de aceitação

É praticamente impossível para um desenvolvedor de software prever como o cliente realmente usará um programa.

Conduzido pelo usuário e não por engenheiros de software, um teste de aceitação pode variar desde um test-drive informal até uma série de testes planejados e sistematicamente executados.

Na verdade, um teste de aceitação pode ser executado por um período de semanas ou meses, descobrindo, assim, erros cumulativos que poderiam degradar o sistema ao longo do tempo.

Testes de aceitação

Muitos construtores de software usam um processo chamado de **teste alfa** e **beta** para descobrir erros que somente o usuário parece ser capaz de encontrar.

O **teste alfa** é realizado no ambiente do desenvolvedor por um grupo representativo de usuários finais.

O software é usado em um cenário natural, com o desenvolvedor “espiando por cima dos ombros” dos usuários, registrando os erros e os problemas de uso.

O **teste beta** é realizado nas instalações de um ou mais usuários finais.

Diferentemente do teste alfa, o desenvolvedor geralmente não está presente.

Portanto, o teste beta é uma aplicação “ao vivo” do software em um ambiente que não pode ser controlado pelo desenvolvedor.

Testes de aceitação

O cliente registra todos os problemas (reais ou imaginários) encontrados durante o teste beta e relata esses problemas para o desenvolvedor em intervalos regulares.

Como resultado dos problemas relatados durante o teste beta, os engenheiros de software fazem modificações e então preparam a liberação do software para todos os clientes.

Teste de sistema

No final, o software é incorporado a outros elementos do sistema (por exemplo, hardware, pessoas, informações), e é executada uma série de testes de integração de sistema e validação.

Tem por objetivo verificar se todos os elementos do sistema foram integrados adequadamente e realizam corretamente suas funções.

Teste de recuperação

Muitos sistemas de computador devem se **recuperar de falhas** e retomar o processamento em pouco ou nenhum tempo de parada.

Em alguns casos, um sistema tem de ser **tolerante a falhas**; ou seja, falhas no processamento não devem causar a paralisação total do sistema.

Em outros casos, uma falha no sistema deve ser corrigida dentro de determinado período de tempo; caso contrário, poderão ocorrer sérios prejuízos financeiros.

Teste de recuperação

Teste de recuperação é um teste do sistema que obriga o software a falhar de várias formas e verifica se a recuperação é executada corretamente.

Se a recuperação for automática (executada pelo próprio sistema), a reinicialização, os mecanismos de verificação, recuperação de dados e reinício são avaliados quanto à correção.

Se a recuperação exige intervenção humana, o tempo médio de reparo (MTTR, mean-time-to-repair) é avaliado para determinar se está dentro dos limites aceitáveis.

Teste de segurança

Qualquer sistema de computador que trabalhe com informações sigilosas ou que cause ações que podem inadequadamente prejudicar (ou beneficiar) indivíduos é um alvo para acesso impróprio ou ilegal.

O teste de segurança tenta verificar se os mecanismos de proteção incorporados ao sistema vão de fato protegê-lo contra acesso indevido.

Com tempo e recursos suficientes, um bom teste de segurança finalmente conseguirá invadir o sistema.

O papel do criador do sistema é tornar o custo da invasão maior do que o valor das informações que poderiam ser obtidas.

Teste por esforço

Teste por esforço

As etapas anteriores de teste resultam em uma avaliação completa das funções normais do programa e do desempenho.

Os testes por esforço (*stress*) servem para colocar os programas em situações anormais.

Basicamente, o testador que executa teste por esforço pergunta:

- “Até onde podemos forçar o sistema até que ele falhe?”.

Teste por esforço

O teste por esforço usa um sistema de maneira que demande recursos em quantidade, frequência ou volumes anormais.

Por exemplo:

1. a taxa de entrada de dados pode ser aumentada por certa magnitude para determinar como as funções de entrada responderão,
2. são executados casos de teste que exigem o máximo de memória ou outros recursos,
3. são criados casos de teste que podem causar excessiva procura por dados residentes em disco.

Basicamente, o testador tenta quebrar o programa.

Teste por esforço

Uma variação do teste por esforço é uma técnica chamada teste de sensibilidade.

Em algumas situações (as mais comuns ocorrem em algoritmos matemáticos), um intervalo muito pequeno de dados, contido dentro dos limites de dados válidos para um programa, pode causar um processamento extremo e até mesmo errôneo ou uma profunda degradação do desempenho.

O teste de sensibilidade tenta descobrir combinações de dados dentro de classes de entrada válidas que possam causar instabilidade ou processamento inadequado.

Teste de desempenho

Para sistemas em tempo real e embutidos, um software que execute a função necessária, mas não esteja em conformidade com os requisitos de desempenho, é inaceitável.

O teste de desempenho é projetado para testar o desempenho em **tempo de execução do software** dentro do contexto de um sistema integrado.

O teste de desempenho é feito em todas as etapas no processo de teste. No entanto, o verdadeiro desempenho de um sistema só pode ser avaliado depois que todos os elementos do sistema estiverem totalmente integrados.

Teste de disponibilização

Teste de disponibilização

Em muitos casos, o software deve operar em uma variedade de plataformas e sob mais de um ambiente de sistema operacional.

O teste de disponibilização, também chamado de teste de configuração, exercita o software em cada ambiente no qual ele deve operar.

Além disso, o teste de disponibilização examina todos os procedimentos de instalação e software de instalação especializado (por exemplo, os “instaladores”) que serão usados pelos clientes e toda a documentação que será usada para fornecer o software aos usuários finais.

A arte da depuração

A arte da depuração

O teste de software é um processo que pode ser sistematicamente planejado e especificado.

A depuração ocorre como consequência de um teste bem-sucedido. Isto é, quando um caso de teste descobre um erro, a depuração é o processo que resulta na remoção do erro.

Embora a depuração possa e deva ser um processo ordenado, ela ainda é, em grande parte, uma arte.

A arte da depuração

Um engenheiro de software, avaliando os resultados de um teste, é frequentemente confrontado com uma indicação sintomática de um problema de software.

Isso significa que a manifestação externa do erro e sua causa interna podem não ter nenhuma relação óbvia uma com a outra. O processo mental mal compreendido que conecta um sintoma a uma causa é a depuração.

O processo de depuração

O processo de depuração

A depuração não é teste, mas frequentemente ocorre em consequência do teste.

O processo de depuração

De acordo com a figura, o processo de depuração começa com a execução de um caso de teste.

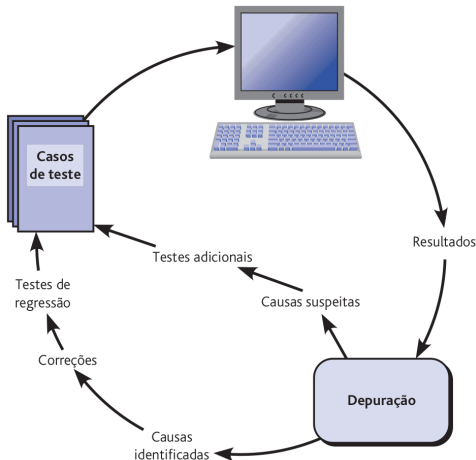


Figura 8: Processo de depuração.

O processo de depuração

Por que a depuração é tão difícil?

O processo de depuração

1. O sintoma e a causa podem estar em componentes "distantes". Componentes altamente acoplados pioram essa situação.
2. O sintoma pode desaparecer (temporariamente) quando outro erro for corrigido.
3. O sintoma pode ser, na realidade, causado por coisas que não são erros (por exemplo, imprecisões de arredondamento).
4. O sintoma pode ser causado por erro humano que não é facilmente rastreado.
5. O sintoma pode ser resultado de problemas de temporização e não de problemas de processamento.
6. Pode ser difícil reproduzir as condições de entrada com precisão.
7. O sintoma pode ser intermitente. Isso é particularmente comum em sistemas embarcados que acoplam hardware e software inextricavelmente.
8. O sintoma pode ocorrer devido a causas distribuídas por várias tarefas, executando em diferentes processadores.

O processo de depuração

Durante a depuração, encontramos erros que variam desde levemente desagradáveis (por exemplo, um formato de saída incorreto) até catastróficos (por exemplo, o sistema falha, causando sérios danos econômicos ou físicos).

À medida que as consequências de um erro aumentam, a pressão para encontrá-lo também aumenta. Às vezes, a pressão obriga um desenvolvedor de software a corrigir um erro e ao mesmo tempo introduzir outros dois.

Considerações psicológicas

Infelizmente, parece haver certa evidência de que a destreza na depuração é uma peculiaridade humana inata.

Algumas pessoas são boas nisso e outras não.

Embora a evidência experimental em depuração seja aberta a muitas interpretações, é possível observar uma grande variação na habilidade de depuração entre programadores com o mesmo nível de formação e experiência. Embora possa ser difícil “aprender” a depurar, várias abordagens do problema podem ser propostas.

Estratégias de depuração

Independentemente da abordagem adotada, a depuração tem um objetivo primordial:

- encontrar e corrigir a causa de um erro ou defeito de software.

O objetivo é alcançado por uma combinação de avaliação sistemática, intuição e sorte.

Estratégias de depuração

Em geral, foram propostas três estratégias de depuração⁴:

- força bruta,
- rastreamento e
- eliminação da causa.

Cada uma dessas estratégias pode ser conduzida manualmente, mas as modernas ferramentas de depuração podem tornar o processo muito mais eficaz.

⁴Myers, G., The Art of Software Testing, Wiley, 1979.

Táticas de depuração. A categoria força bruta para depuração é provavelmente o método mais comum e menos eficiente para isolar a causa de um erro de software.

Usamos métodos de depuração do tipo força bruta quando todo o resto falha.

Estratégias de depuração

Usando uma filosofia do tipo **deixe o computador encontrar o erro**, ocorrem despejos de memória, rastreamentos em tempo de execução, e o programa é sobrecarregado com instruções de saída.

Espera-se que, no meio de toda aquela confusão de informações produzidas, consigamos encontrar um indício que possa levar à causa de um erro.

Embora a grande quantidade de informações produzidas possa, no fim, levar ao sucesso, mais frequentemente é uma perda de tempo e trabalho.

É preciso pensar primeiro!

Rastreamento (backtracking) é uma abordagem comum de depuração que pode ser usada com sucesso em programas pequenos.

Começando no ponto onde o sintoma foi descoberto, o código-fonte é investigado retroativamente (manualmente) até que a causa seja encontrada.

Infelizmente, à medida que o número de linhas de código-fonte aumenta, o número de caminhos retroativos em potencial pode se tornar demasiadamente grande.

A terceira abordagem de depuração – eliminação da causa – é manifestada por indução ou dedução e introduz o conceito de particionamento binário.

Os dados relacionados à ocorrência do erro são organizados para isolar as possíveis causas.

Estratégias de depuração

É imaginada uma “hipótese de causa”, e os dados mencionados anteriormente são usados para provar ou negar a hipótese.

Como alternativa, é preparada uma lista de todas as causas possíveis, e são realizados testes para eliminar cada uma delas.

Se os testes iniciais indicam que determinada causa hipotética promete resultado, os dados são refinados tentando **isolar o defeito**.

Depuração automática. Cada um desses métodos de depuração pode ser complementado com ferramentas de depuração que podem fornecer suporte semiautomático para o engenheiro de software à medida que são tentadas as estratégias.

Ambientes Integrados de Desenvolvimento (IDE) proporcionam uma maneira de capturar alguns dos erros predeterminados, específicos da linguagem (por exemplo, falta de caracteres de fim de instrução, variáveis indefinidas, e assim por diante), sem exigir compilação".

Há disponível uma ampla variedade de compiladores de depuração, auxílios dinâmicos de depuração (“rastreadores”), geradores de casos de teste automáticos e ferramentas de mapeamento de referências cruzadas.

No entanto, as ferramentas não substituem uma avaliação cuidadosa, fundamentada em um modelo completo de projeto e um código-fonte claro.

O fator humano. Qualquer discussão sobre abordagens e ferramentas de depuração é incompleta se não mencionar um poderoso aliado: outras pessoas! Um ponto de vista novo, não ofuscado por horas de frustração, pode fazer maravilhas.

Uma máxima final para a depuração seria:

- “Quando tudo mais falhar, procure ajuda!”.

Correção do erro

Uma vez encontrado um erro, ele precisa ser corrigido.

Entretanto, conforme já observamos, a correção de um erro pode introduzir outros erros e, portanto, causar mais danos do que trazer benefícios.

Van Vleck⁵ sugere três perguntas simples que você deve fazer antes de fazer a “correção” que remove a causa de um erro:

1. A causa do defeito é reproduzida em alguma outra parte do programa?
2. Qual “próximo erro” poderia ser introduzido pela correção que estou prestes a fazer?
3. O que poderíamos ter feito para evitar esse defeito já no início?

⁵Van Vleck, T., “Three Questions About Each Bug You Find”, ACM Software Engineering Notes, vol. 14, no. 5, July 1989, pp. 62–63.

Referências

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides.
Padrões de Projetos: Soluções Reutilizáveis de Software Orientados a Objetos, volume 1.
Bookman, 1. edition, 2000.
- [2] R. S. Pressman and B. R. Maxim.
Engenharia de Software uma abordagem profissional, volume 1.
AMGH Editora Ltda, 9. edition, 2021.
- [3] S. R. Schach.
Engenharia de Software: Os paradigmas clássicos & orientados a objetos, volume 1.
McGraw-Hill, 7. edition, 2008.
- [4] I. Sommerville.
Engenharia de Software, volume 1.
São Paulo: Addison-Wesley, 10. edition, 2019.

- [5] M. T. Valente.
Engenharia de software moderna, volume 1.
Independente, 2020.
- [6] R. S. Wazlawick.
Engenharia de Software - Conceitos e Prática, volume 1.
Rio de Janeiro: GEN LTC, 2. edition, 2019.

Perguntas?

Obrigado pela atenção!